

Positive Edge Triggered Negative Edge Triggered Reset D Flip-Flop

Truth table

Positive Edge Triggered Negative Edge Triggered Reset D Flip-Flop (X=0,1)			
rst	clk	data	q
1		X	X
1		X	previous output
0		X	0

Behavioral source code (penr_dff.v)

```
module penr_dff(q,data,rst,clk);
output reg q;
input data,rst,clk;
always @(posedge clk or negedge rst)
begin
    if(rst==0)
        q<=1'b0;
    else
        q<=data;
end
endmodule
```

Testbench source code (penr_dff_tb.v)

```
`timescale 1ns/1ps

module penr_dff_tb;
    // 1. Creating registers for inputs and a wire for the output
    reg test_clk;
    reg test_rst;
    reg test_data;
    wire test_q;

    // 2. Instantiating the actual D Flip Flop module (connect pins)
    penr_dff uut (
        .q(test_q),
        .clk(test_clk),
        .rst(test_rst),
        .data(test_data)
    );
endmodule
```

```
// 3. Generating a clock, test_clk changes every 1 ns, so time-period of clock = 2 ns, frequency = 0.5 GHz
always #1 test_clk = ~test_clk;

// 4. Generating the input signals over time
initial begin
    // Telling iverilog to dump waveform data for GTKWave
    $dumpfile("simulation.vcd");
    $dumpvars(0, penr_dff_tb);

// 5. Initial condition
test_clk = 0;
test_rst = 0;

// Test Case 1: while D Flip Flop is in asynchronous reset, setting data = 0 and then data = 1
test_data = 0; #5.5
test_data = 1; #5.5

// Test Case 2: Lifting reset & setting data = 1 before a rising clock edge
test_rst = 1; #5.5;
test_data = 1; #1;

// Test Case 3: Changing data to 0 before the next rising clock edge
test_data = 0; #5.5;

// Test Case 4: Setting data = 1 again
test_data = 1; #3;

// Test Case 5: Resetting instantly while clock is running
test_rst = 0; #10

// Test Case 6: While reset is active, let's give data = 1
test_data = 1;

$finish; // End simulation
end
endmodule
```

Opening the OSS CAD Suite environment

```
call C:\oss-cad-suite\environment.bat
```

Compiling the design & testbench files

```
iverilog -o simulation.vvp penr_dff.v penr_dff_tb.v
```

Running the compiled simulation

```
vvp simulation.vvp
```

Generating & checking output waveform - Fig. 1(a)

```
gtkwave simulation.vcd
```

Starting the Yosys synthesis tool

```
yosys
```

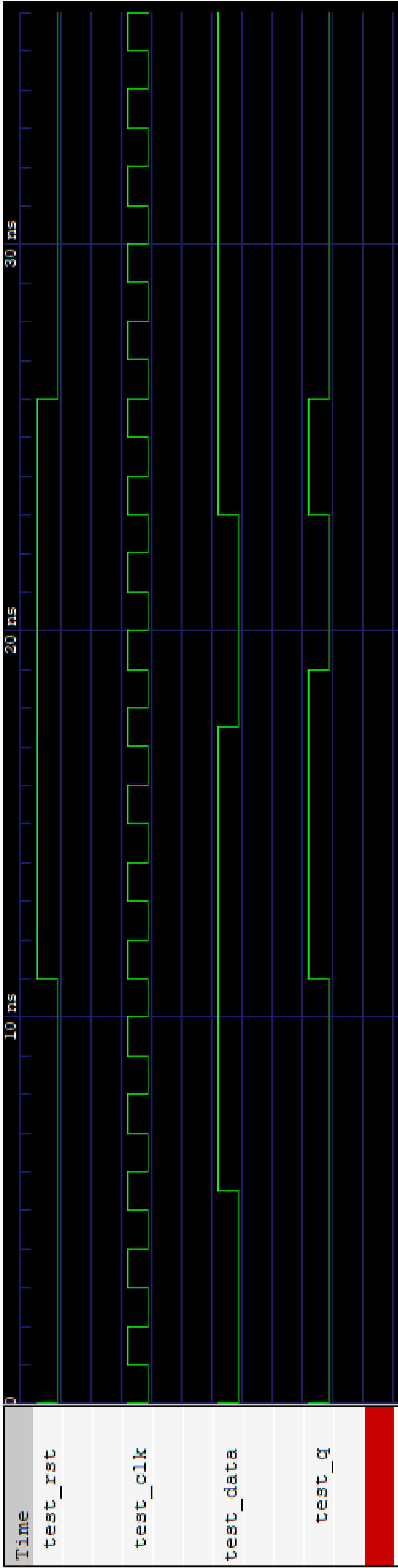


Fig. N(a) - +ET -ET Reset D Flip-Flop output waveform

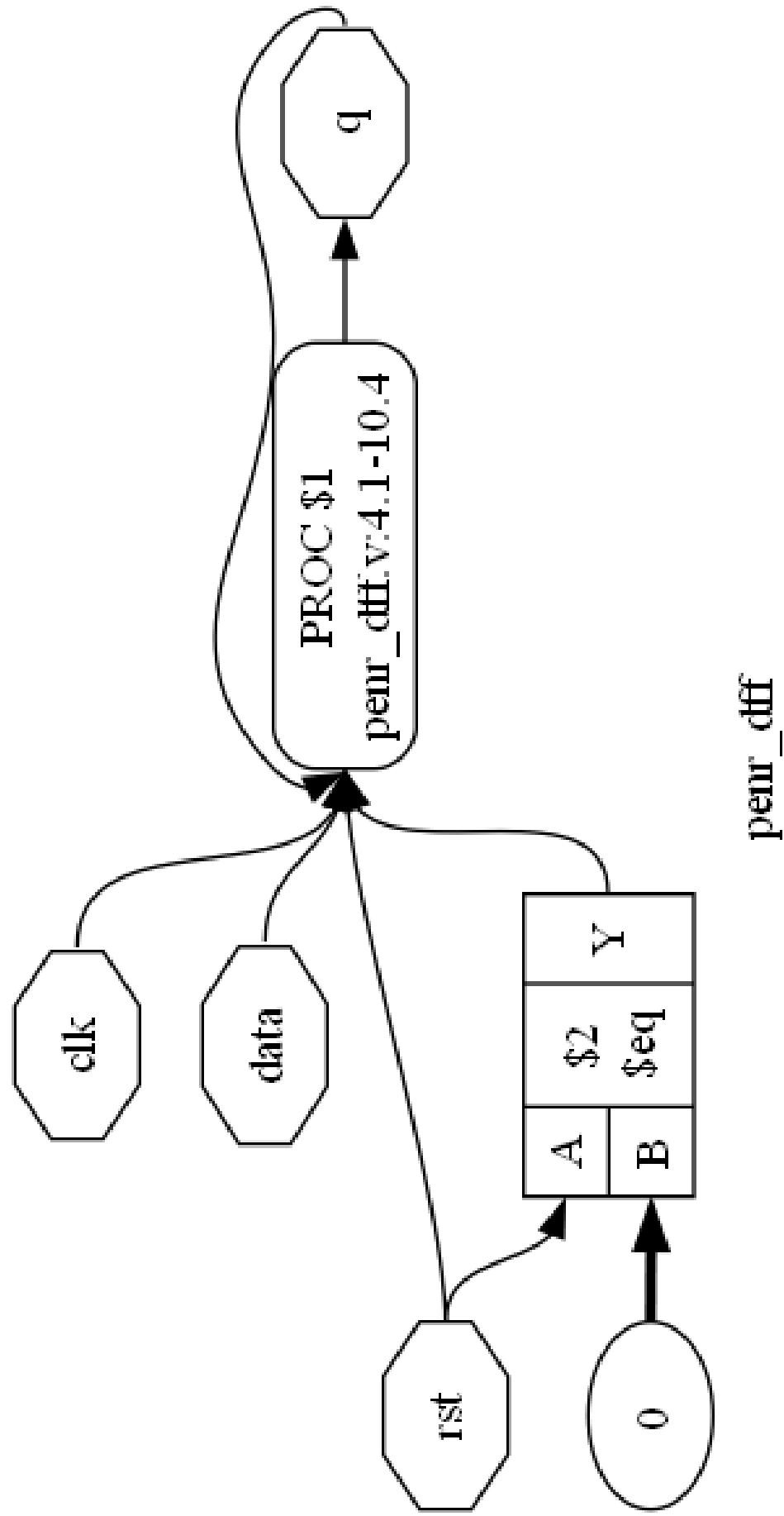


Fig. N(b) - +ET -ET Reset D Flip-Flop generated schematic

```

/-----\
| yosys -- Yosys Open SYnthesis Suite                               |
| Copyright (C) 2012 - 2026  Claire Xenia Wolf <claire@yosyshq.com> |
| Distributed under an ISC-like license, type "license" to see terms |
\-----/
Yosys 0.68+40 (git sha1 0f2bcb94b-dirty, Release, GNU /usr/bin/x86_64-w64-mingw32-g++ 15.2.1)

```

Reading the behavioral source file

```
yosys> read_verilog penr_dff.v
```

Setting top-level module for synthesis

```
yosys> hierarchy -top penr_dff
```

Converting behavioral processes, optimizing logic, mapping to technology primitives, and optimizing again

```
yosys> proc opt techmap opt
```

Writing the synthesized Verilog netlist

```
yosys> write_verilog penr_dff_netlist.v
```

Generating the synthesized schematic

```
yosys> show
```

```

5. Generating Graphviz representation of design.
Writing dot description to 'show.dot'.
Dumping module penr_dff to page 1.

```

Exiting the Yosys tool

```
yosys> exit
```

```

Warnings: 2 unique messages, 3 total
End of script. Logfile hash: a730de9a18
Yosys 0.68+40 (git sha1 0f2bcb94b-dirty, Release, GNU /usr/bin/x86_64-w64-mingw32-g++ 15.2.1)
Time spent: 5% 2x write_verilog (0 sec), 5% 2x read_verilog (0 sec), ...

```

Generate & check +ET -ET Reset D Flip-Flop schematic - Fig. N(b)

```

[OSS CAD Suite] D:\[redacted] \NOT>"D:\[redacted]
15.1.1-win64\bin\dot.exe" -Tpng show.dot -o schematic.png

```

